COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/30/2026	**DATE SUBMITTED:**	8/11/2026
NAME:	Cedric S. Fernandez	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 **[Linked Lists]**

TABLE 3-1 Output of Initial/Simple Implementation

CODE

TRAVERSAL

```
13  template <typename T>
14  void ListTraversal(SingleList<T>* head){
15
16      if (head == nullptr){
17          std::cout << "The list is empty";
18          return;
19      }
20
21      while(head != nullptr){
22          // PRINT data OF n
23          std::cout<<head->data;
24
25          if(head->next != nullptr){
26              std::cout << "->";
27          }
28
29          // GO TO NEXT NODE n -> next
30          head = head->next;
31      }
32
33      std::cout << std::endl;
34  }
35
```

INSERTION HEAD

```
36  template <typename T>
37  void sllinsertHead(T newData, SingleList<T>** head){
38
39      // 1. Allocate new memory for the new code
40      SingleList<T>* newNode = new SingleList<T>;
41      // 2. Put our data into the new node
42      newNode->data = newData;
43      // 3. Set next to the new node to point to the previous head
44      newNode->next = *head;
45      // 4. Reset head to the new node
46      *head = newNode;
47  }
48
```

GENERAL INSERT

```
49  template <typename T>
50  void sllGeneralInsert(T newData, SingleList<T>* prevNode){
51
52      //1. Check if it is the head node (previous node is null)
53      if(prevNode == nullptr){
54          //2. If null, print "Previous node cannot be null."
55          std::cout<<"Previous value cannot be null"<<std::endl;
56          return;
57      }
58      //3. Allocate a new node
59      SingleList<T>* newNode = new SingleList<T>;
60      //4. Store data in the new node
61      newNode->data = newData;
62      //5. Point new node to the node previous node was pointing to
63      newNode->next = prevNode->next;
64      //6. Point previous node to the new node
65      prevNode->next = newNode;
66  }
67
```

INSERT AT END

```
68  template <typename T>
69  void sllinsertEnd(T newData, SingleList<T>** head){
70      // 1. Allocate a new node
71      SingleList<T>* newNode = new SingleList<T>;
72
73      // 3. Store data in new node
74      newNode->data = newData;
75      // 4. Point to null
76      newNode->next = nullptr;
77
78      // 2. Dereference the head node
79      SingleList<T>* currentNode = *head;
80
81      // 5. Traverse the list
82      while(currentNode->next != nullptr){
83          currentNode = currentNode->next;
84      }
85
86      // 6. Point the next of the current node to the new node
87      currentNode->next = newNode;
88  }
89
```

DELETING NODE

```
90  template <typename T>
91  void sllDeleteNode(T findData, SingleList<T>** head){
92      // if there is nothing to delete
93      if (*head == nullptr) return;
94      SingleList<T>* currNode = *head;
95      SingleList<T>* prevNode = nullptr;
96
97      // searching the node: 1. the node does not exist + we found data
98      while (currNode != nullptr && currNode->data != findData){
99          prevNode = currNode;
100         currNode = currNode->next;
101
102     }
103     // if data not found currNode == nullPtr in the while loop
104     if(currNode == nullptr){
105         std::cout << "the data was not found\n" << std::endl;
106         return;
107     }
108     // if data was found
109     if(prevNode == nullptr){
110         *head = currNode->next;
111     } else {
112         prevNode->next = currNode->next;
113     }
114
115     delete currNode;
116 }
```

OUTPUT

```
Testing of traversal
C->E->D->R->K
Testing of sllinsertHead
Q->C->E->D->R->K
Testing of sllGeneralInsert
Q->C->E->X->D->R->K
Testing of sllinsertEnd
Q->C->E->X->D->R->K->U
Testing of deleting the node
Q->C->E->X->D->R->K
Deleting all nodes
The list is empty
```

Observation:

The program demonstrated how Singly in link list works with the implementation of templates along with it showing us that it can allow flexible data storage. It also showed us functions like traversal insertion on the head general insertion insertion by the end deletion of a node and deletion of all nodes making us understand how nodes are created, navigated, linked and also deleted in a memory

Table 3-2. Code for the List Operations

CODE

```
#include <iostream>
#include "singly_ll.h"
#include "doubly_ll.h"


int main(){
    // create list with "C P E"
    DoubleList<char>* head = CreateNewNode('C');
    DoubleList<char>* second = CreateNewNode('P');
    DoubleList<char>* third = CreateNewNode('E');

    head->next = second;
    second->prev = head;
    second->next = third;
    third->prev = second;

    //traversal
    std::cout<<"Testing the DLL Traversal"<<std::endl;
    dllTraverse(head);

    //testing the insert at the head
    std::cout<<"\nTesting the insertion at the head node"<<std::endl;
    dllInsertHead('X', &head);
    dllTraverse(head);

    //test of insertion in the end
    std::cout<<"\nTesting the insertion at the end"<<std::endl;
    dllInsertEnd('0', head);
    dllTraverse(head);

    //Test general insert
    std::cout << "\nTesting of dllGeneralInsert\n";
    dllGeneralInsert('J', head->next);
    dllTraverse(head);

    //testing deletion of node
    std::cout<<"\nTesting of dllDeleteNode\n";
    dllDeleteNode('C', &head);
    dllTraverse(head);

    //Deletion of whole list
    std::cout<<"\nTesting of deletion of whole list\n";
    dllDeleteList(&head);
    dllTraverse(head);
}
```

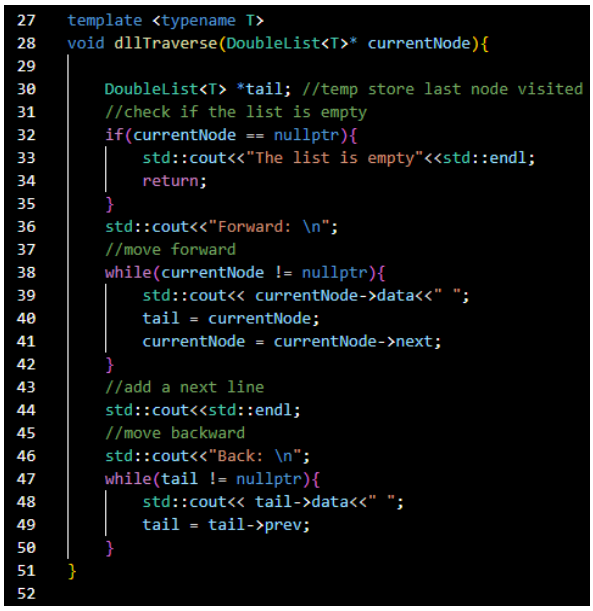
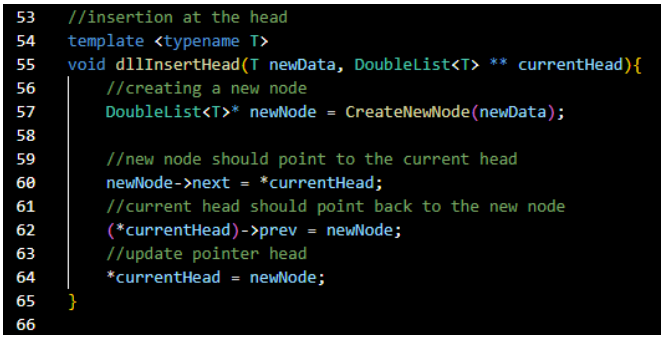
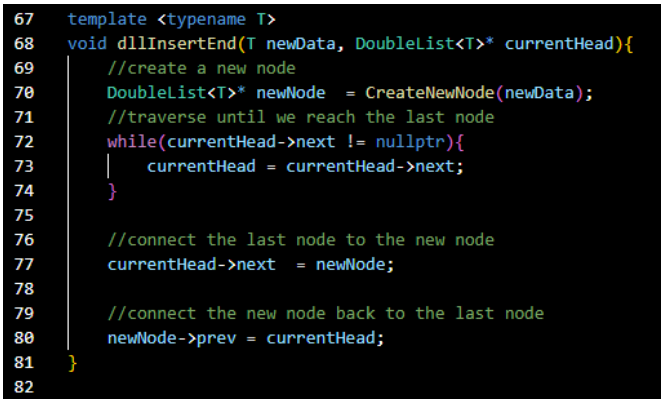
OUTPUT

```
Testing the DLL Traversal
Forward:
C P E
Back:
E P C
Testing the insertion at the head node
Forward:
X C P E
Back:
E P C X
Testing the insertion at the end
Forward:
X C P E 0
Back:
0 E P C X
Testing of dllGeneralInsert
Forward:
X C J P E 0
Back:
0 E P J C X
Testing of dllDeleteNode
Forward:
X J P E 0
Back:
0 E P J X
Testing of deletion of whole list
The list is empty
```

Observation:

This code demonstrates the core mechanics of double linked list unlike the singly it showed that it can maintain both next and prev pointers that enables them to move across any end of the list. It also gave a highlight to the different list operations like inserting at the head, end, middle, and also deleting specific elements and by the end deleting the entire list from the memory.

Table 3-3 Code and Analysis for Singly Linked Lists

CODE	OUTPUT
	This function prints the list from start to finish. Once it reaches the end, it turns around and uses the last item to print everything in reverse, back to the start.
	This function adds a new item to the very beginning of the list. It connects the new item to the old first item, links them in both directions, and marks the new item as the new start.
	This function adds a new item to the very end of the list. It moves through the list until it finds the current last item, attaches the new item behind it, and links them both together so the path stays connected in both directions.

```
27  template <typename T>
28  void dllTraverse(DoubleList<T>* currentNode){
29
30      DoubleList<T> *tail; //temp store last node visited
31      //check if the list is empty
32      if(currentNode == nullptr){
33          std::cout<<"The list is empty"<<std::endl;
34          return;
35      }
36      std::cout<<"Forward: \n";
37      //move forward
38      while(currentNode != nullptr){
39          std::cout<< currentNode->data<<" ";
40          tail = currentNode;
41          currentNode = currentNode->next;
42      }
43      //add a next line
44      std::cout<<std::endl;
45      //move backward
46      std::cout<<"Back: \n";
47      while(tail != nullptr){
48          std::cout<< tail->data<<" ";
49          tail = tail->prev;
50      }
51  }
52
```

```
53  //insertion at the head
54  template <typename T>
55  void dllInsertHead(T newData, DoubleList<T> ** currentHead){
56      //creating a new node
57      DoubleList<T>* newNode = CreateNewNode(newData);
58
59      //new node should point to the current head
60      newNode->next = *currentHead;
61      //current head should point back to the new node
62      (*currentHead)->prev = newNode;
63      //update pointer head
64      *currentHead = newNode;
65  }
66
```

```
67  template <typename T>
68  void dllInsertEnd(T newData, DoubleList<T>* currentHead){
69      //create a new node
70      DoubleList<T>* newNode  = CreateNewNode(newData);
71      //traverse until we reach the last node
72      while(currentHead->next != nullptr){
73          currentHead = currentHead->next;
74      }
75
76      //connect the last node to the new node
77      currentHead->next  = newNode;
78
79      //connect the new node back to the last node
80      newNode->prev = currentHead;
81  }
82
```

```cpp
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* prevNode){
    if(prevNode == nullptr){
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    DoubleList<T>* newNode  = CreateNewNode(newData);
    DoubleList<T>* nextNode;

    newNode->data = newData;
    nextNode = prevNode->next;
    newNode->next = prevNode->next;
    prevNode->next = newNode;
    nextNode->prev = newNode;
    newNode->prev = prevNode;

}
```

This function inserts a new item right after a given item in the middle of the list. It stops automatically if you try to use it on the very first item (since it's only built for middle items), then updates the pointers so the new item sits smoothly between its neighbors.

```cpp
template <typename T>
void dllDeleteNode(T findData, DoubleList<T>** head){
    // if there is nothing to delete
    if (*head == nullptr) return;
    DoubleList<T>* currNode = *head;
    DoubleList<T>* prevNode = nullptr;
    DoubleList<T>* nodeNext;
    // searching the node: 1. the node does not exist + we found data
    while (currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }
    nodeNext = currNode->next;
    // if data not found currNode == nullPtr in the while loop
    if(currNode == nullptr){
        std::cout << "the data was not found\n" << std::endl;
        return;
    }

    // if data was found
    if(prevNode == nullptr){
        *head = currNode->next;
        nodeNext = currNode;
    } else if (nodeNext != nullptr) {
        prevNode->next = currNode->next;
        nodeNext->prev = currNode->prev;
    }
    delete currNode;
}
```

This function searches the list for a specific value. If the item isn't found, it prints a "not found" message and stops. If found, it connects the items before and after it so they skip over the target item, updates the start if the first item was removed, and deletes the target item to clear memory.

COMPUTER ENGINEERING

QUEZON CITY

SUPPLEMENTARY

CODE

INT MAIN

```
#include <iostream>
#include "supplementarysingly_ll.h"
#include <string>


int main(){
    // Creates playlist track nodes with song titles
    musicList<std::string>* head = AddSong(std::string("SWAY - Michael Buble"));
    musicList<std::string>* second = AddSong(std::string("Better with time - Grabbitz"));
    musicList<std::string>* third = AddSong(std::string("Tip Toe - HYBS"));
    musicList<std::string>* fourth = AddSong(std::string("Sunflower - Post Malone, Swae Lee"));
    musicList<std::string>* fifth = AddSong(std::string("YMCA - Village People"));

    // Manually link nodes in both directions to establish playlist order
    head->next = second;
    second->prev = head;
    second->next = third;
    third->prev = second;
    third->next = fourth;
    fourth->prev = third;
    fourth->next = fifth;
    fifth->prev = fourth;
```

OUTPUT

PLAYING PLAYLIST

```
=====Menu=====

1. Play Playlist
2. add Song
3. remove Song
4. Exit
Please enter valid command from (1-4): 1
Music Playlist
SWAY - Michael Buble
Better with time - Grabbitz
Tip Toe - HYBS
Sunflower - Post Malone, Swae Lee
YMCA - Village People
:End of Playlist:
--(Return to Start)--
SWAY - Michael Buble
```

ADDING SONG

```
=====Menu=====

1. Play Playlist
2. add Song
3. remove Song
4. Exit
Please enter valid command from (1-4): 2
Song Name: Baby by Justin Bieber

=====Menu=====

1. Play Playlist
2. add Song
3. remove Song
4. Exit
Please enter valid command from (1-4): 1
Music Playlist
SWAY - Michael Buble
Better with time - Grabbitz
Tip Toe - HYBS
Sunflower - Post Malone, Swae Lee
YMCA - Village People
Baby by Justin Bieber
:End of Playlist:
--(Return to Start)--
SWAY - Michael Buble
```

```
    int options = 0;
    // Interactive menu loop
    while(options <= 3){
     std::cout<<"=====Menu=====\n\n";

     std::cout<< "1. Play Playlist" <<
std::endl;
     std::cout<< "2. Insert Song" <<
std::endl;
     std::cout<< "3. Add Song" <<
std::endl;
     std::cout<< "4. Exit" <<
std::endl;

     std::cout<<"Please enter valid
command from (1-4): ";
     std::cin>>options;
     switch(options){
        case 1:{
           // Option 1: Display and
traverse all tracks in the playlist
           playlistTraverse(head);
           std::cout<<std::endl;
           break;
        }
        case 2:{
           // Option 2: Get song title
from user input and append it to the
end
           std::cout<<"Song Name: ";
           std::string insertSong;
           std::cin>>std::ws; // Clear
whitespace
           std::cout<<std::endl;
           std::getline(std::cin,
insertSong);
```

REMOVING SONG

```
=====Menu=====

1. Play Playlist
2. add Song
3. remove Song
4. Exit
Please enter valid command from (1-4): 3
Song Name: YMCA - Village People

=====Menu=====

1. Play Playlist
2. add Song
3. remove Song
4. Exit
Please enter valid command from (1-4): 1
Music Playlist
SWAY - Michael Buble
Better with time - Grabbitz
Tip Toe - HYBS
Sunflower - Post Malone, Swae Lee
Baby by Justin Bieber
:End of Playlist:
--(Return to Start)--
SWAY - Michael Buble
```

EXIT PROGRAM

```
=====Menu=====

1. Play Playlist
2. add Song
3. remove Song
4. Exit
Please enter valid command from (1-4): 4
PS C:\Users\csf18\Desktop\HOA.3\SUPPLEMENTARY>
```

```
playlistInsertEnd(insertSong, head);
            break;
        }
        case 3:{
            // Option 3: Get target song
title and search/remove it from
playlist
            std::cout<<"Song Name: ";
            std::string removeSong;
            std::cin>>std::ws; // Clear
whitespace
            std::cout<<std::endl;
            std::getline(std::cin,
removeSong);
            deleteSong(removeSong,
&head);
            break;
         }
         case 4:{
            // Option 4: Exit menu loop
            break;
         }
    }
 }
}
```

SUPPLEMENTARY SINGLY

```
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

// Represents an individual track/node
in the music playlist
template <typename T>
class musicList{
    public:
       T data;
       musicList<T>* prev = nullptr;
```

```
        musicList<T>* next = nullptr;
};

// Helper function to create and set up
a new song node
template <typename T>
musicList<T> *AddSong(T newData){
    // Allocate heap memory for the new
track
    musicList<T> *newNode = new
musicList<T>;
    // Store song details into the node
    newNode->data = newData;
    // Initialize adjacent track
pointers to null
    newNode->next = nullptr;
    newNode->prev = nullptr;
    // Return the memory address of the
newly added track

    return newNode;
}

// Display all songs in the playlist
from start to finish
template <typename T>
void playlistTraverse(musicList<T>*
currentNode){

    musicList<T> *headNode; // Store
reference to the first track
    // stop if the playlist contains no
tracks
    if(currentNode == nullptr){
        std::cout<<"The list is
empty"<<std::endl;
        return;
```

```
    }
    headNode = currentNode;
    // Iterate forward through the
playlist
    std::cout<<"Music
Playlist"<<std::endl;
    while(currentNode != nullptr){
        std::cout<<
currentNode->data<<" \n";
        currentNode =
currentNode->next;
    }
    std::cout<<":End of
Playlist:"<<std::endl;
    if(currentNode == nullptr){
        std::cout<< "--(Return to
Start)--\n"<< headNode->data<<"\n";
    }
}

// Append a new song to the end of the
playlist
template <typename T>
void playlistInsertEnd(T newData,
musicList<T>* currentHead){
    // Create the new song node
    musicList<T>* newNode  =
AddSong(newData);
    // Traverse forward until reaching
the current last song
    while(currentHead->next !=
nullptr){
        currentHead =
currentHead->next;
    }

    // Link the old end track forward
```

```
to the new song
    currentHead->next  = newNode;

    // Link the new song backward to
the old end track
    newNode->prev = currentHead;
}

// Search for and remove a specific
song from the playlist
template <typename T>
void deleteSong(T findData,
musicList<T>** head){

    // Exit immediately if playlist is
empty
    if (*head == nullptr) return;

    musicList<T>* currNode = *head;
    musicList<T>* prevNode = nullptr;

    // Search for target track matching
findData
    while (currNode != nullptr &&
currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;

    }

    // Song target was not present in
the playlist
    if(currNode == nullptr){
        std::cout << "the data was not
found\n" << std::endl;
        return;
    }
```

```
    // Re-link playlist around target
node
    if(prevNode == nullptr){
        *head = currNode->next; //
Removing the first song in playlist
    } else {
        prevNode->next =
currNode->next; // Skipping over
deleted song
    }

    delete currNode; // Free memory for
deleted track
}

#endif
```

CONCLUSION

Through this supplementary activity, I learned how to apply linked list concepts to a practical application by building an interactive music playlist manager. The procedure involved dynamically allocating memory for song nodes, linking pointers and managing list through functions for track traversal, insertion, and deletion.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of content, Grammar correction, Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used to explain function concepts and how to properly use them, along with correcting my grammar for proper explanation and how to do circular Linked list.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.